

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Atividade 2:

Atividade 2 – Ingestão e ETL com linguagem de programação (Python)

Utilizar linguagem de programação Python para ingestão e tratamento de dados.

- Pacotes adicionais podem ser utilizados (exceto Spark, DuckDB)
- Tratamento de dados não deve ser realizado via SQL

Realizar a ingestão de todas as base de dados em um banco de dados relacional open source. Pode ser utilizado qualquer banco de dado sendo algumas sugestões:

- MySQL
- Postgre
- ClickHouse

Gerar uma tabela final com os dados tratados e unidos.

- O tratamento de dados deve ser realizado através da linguagem de programação Python

Adicionar as seguintes camadas de processamento, dentro do próprio banco de dados ou em disco local. A Camada Delivery deve obrigatoriamente ter estar também no formato de uma tabela final dentro do banco de dados relacional:

- RAW – formato dos dados livre
- Trusted – formato de dados em Parquet ou ORC or AVRO (indicado Parquet)
- Delivery – formato de dados em Parquet ou ORC or AVRO (indicado Parquet)

Infraestrutura

Container PostgreSQL criado e rodando via Docker:

```
(base) camil@MacBook-Air atividade_02 % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
6af21bae8fb4   postgres:16   "docker-entrypoint.s..." 49 minutes ago Up 20 minutes 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp pg-eedb022-a2
(base) camil@MacBook-Air atividade_02 %
```

Estrutura de pastas criada:

```
((base) camil@MacBook-Air atividade_02 % tree
.
├── data
│   └── raw
│       ├── Bancos
│       │   └── EnquadramentoInicia_v2.tsv
│       ├── Empregados
│       │   ├── glassdoor_consolidado_join_match_less_v2.csv
│       │   └── glassdoor_consolidado_join_match_v2.csv
│       └── Reclamacoes
│           ├── 2021_tri_01.csv
│           ├── 2021_tri_02.csv
│           ├── 2021_tri_03.csv
│           ├── 2021_tri_04.csv
│           ├── 2022_tri_01.csv
│           ├── 2022_tri_02_nao_ha_dados.csv
│           ├── 2022_tri_03.csv
│           └── 2022_tri_04.csv
├── Readme.md
└── scripts
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Script de conexão com o banco:

```
eEDB-022 > atividade_02 > scripts >  conexao.py > ...
1  import os
2  from sqlalchemy import create_engine
3
4  DB_HOST = os.getenv("DB_HOST", "localhost")
5  DB_PORT = os.getenv("DB_PORT", "5433")
6  DB_NAME = os.getenv("DB_NAME", "eedb022_a2")
7  DB_USER = os.getenv("DB_USER", "postgres")
8  DB_PASSWORD = os.getenv("DB_PASSWORD", "camila123")
9
10
11 def get_engine():
12     connection_string = f"postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
13     engine = create_engine(connection_string)
14     return engine
15
16
17 if __name__ == "__main__":
18     engine = get_engine()
19     try:
20         with engine.connect() as conn:
21             print("Conexão com o banco funcionando :)")
22     except Exception as e:
23         print(f"Erro ao conectar: {e}")
24
```

Conexão bem-sucedida:

```
[(base) camil@MacBook-Air atividade_02 % python scripts/conexao.py
Conexão com o banco funcionando :)
(base) camil@MacBook-Air atividade_02 %
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

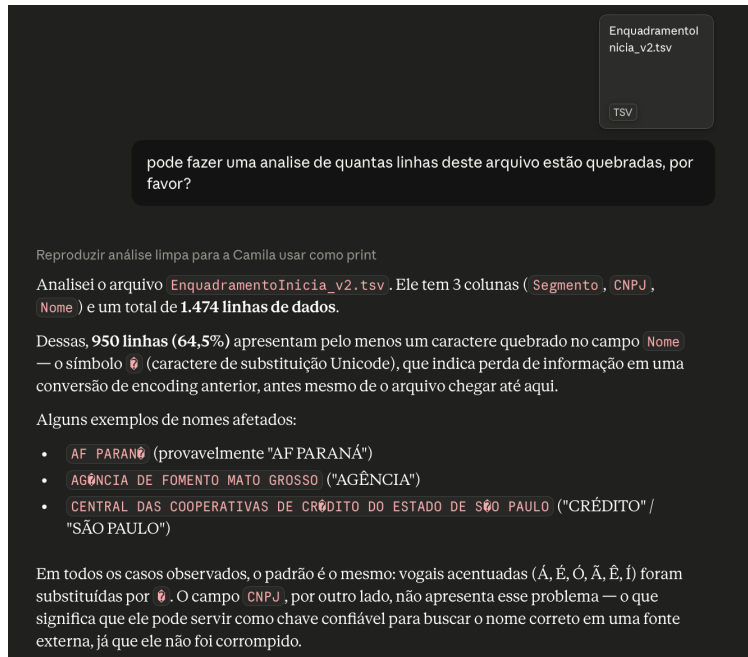
Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Bancos

Início do tratamento do arquivo EnquadramentoInicia_v2.tsv (possui varias linhas com caracteres quebrados)

Uso de IA para mensurar o problema:



Para tentar solucionar esse problema, tentaremos usar a API p blica do Banco Central (IFData, endpoint `IfDataCadastro`). A requisi  o traz o cadastro de institui  es financeiras (nome oficial e CNPJ), filtrado por m s de refer ncia. O cruzamento ser  feito pelo campo CNPJ, que n o estava corrompido no arquivo original, permitindo substituir os nomes quebrados pelos nomes oficiais, utilizando um de-para.

Script para testar conex  o com a API:

```
eEDB-022 > atividade_02 > scripts > consulta_bcb.py > ...
1 import requests
2
3 url = "https://olinda.bcb.gov.br/olinda/servico/IFDATA/versao/v1/odata/IfDataCadastro(AnoMes=@AnoMes)?@AnoMes=202412&$format=json&$top=3"
4
5 response = requests.get(url)
6 print(response.status_code)
7 print(response.json())
```

Sucesso no teste:

```
(base) camil@MacBook-Air atividade_02 % python scripts/consulta_bcb.py
200
{'@odata.context': 'https://was-p.bcbnet.bcb.gov.br/olinda/servico/IFDATA/versao/v1/odata/$metadata#IfDataCadastro', 'value': [{'CodInst': 'C0041856', 'Data': '202412', 'NomeInstituicao': 'ABC-BRASIL', 'DataInicioAtividade': 180001, 'Tcb': 'B1', 'Td': 'C', 'Tc': 3, 'SegmentoTb': None, 'Atividade': None, 'Uf': 'SP', 'Municipio': 'S o Paulo', 'Sr': 'S3', 'CodConglomeradoFinanceiro': 'C0041856', 'CodConglomeradoPrudencial': 'C0080312', 'CnpjInstituicaoLider': '28195667', 'Situacao': 'A'}, {'CodInst': 'C0080312', 'Data': '202412', 'NomeInstituicao': 'ABC-BRASIL - PRUDENCIAL', 'DataInicioAtividade': 180001, 'Tcb': 'B1', 'Td': 'C', 'Tc': 3, 'SegmentoTb': None, 'Atividade': 'Cr dito Atacado', 'Uf': 'SP', 'Municipio': 'S o Paulo', 'Sr': 'S3', 'CodConglomeradoFinanceiro': 'C0041856', 'CodConglomeradoPrudencial': 'C0080312', 'CnpjInstituicaoLider': '28195667', 'Situacao': 'A'}, {'CodInst': 'C0052302', 'Data': '202412', 'NomeInstituicao': 'ABN AMRO', 'DataInicioAtividade': 180001, 'Tcb': 'B1', 'Td': 'C', 'Tc': 3, 'SegmentoTb': None, 'Atividade': None, 'Uf': 'SP', 'Municipio': 'S o Paulo', 'Sr': 'S4', 'CodConglomeradoFinanceiro': 'C0052302', 'CodConglomeradoPrudencial': 'C0081579', 'CnpjInstituicaoLider': '03532415', 'Situacao': 'A'}]}
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Primeira tentativa de montar o de-para:

```
eEDB-022 > atividade_02 > scripts > monta_depara_bcb.py > ...
1  import requests
2  import time
3
4  url = "https://olinda.bcb.gov.br/olinda/servico/IFDData/versao/v1/odata/IfDataCadastro(AnoMes=@AnoMes)?@AnoMes=202412&$format=json&$top=10000"
5
6  response = requests.get(url)
7  dados = response.json()["value"]
8
9  print(f"Total de registros recebidos: {len(dados)}")
10 print(dados[0])
```

Deu certo! Nosso de-para conta com 5744 registros, bem mais que os cerca de 2 mil na nossa tabela, o que nos aproxima mais ainda da possibilidade de conseguirmos encontrar os nomes corretos dos bancos que buscarmos

```
(base) camil@MacBook-Air atividade_02 % python scripts/monta_depara_bcb.py
Total de registros recebidos: 5744
{'CodInst': 'C0041856', 'Data': '202412', 'NomeInstituicao': 'ABC-BRASIL', 'DataInicioAtividade': 180001, 'Tcb': 'B1', 'Td': 'C',
'Tc': 3, 'SegmentoTb': None, 'Atividade': None, 'Uf': 'SP', 'Municipio': 'São Paulo', 'Sr': 'S3', 'CodConglomeradoFinanceiro': 'C0041856', 'CodConglomeradoPrudencial': 'C0080312', 'CnpjInstituicaoLider': '28195667', 'Situacao': 'A'}
```

Adicionando bloco final para criar de fato o de-para e salvar ele em json, para ser lido pelo script de tratamento da tabela de bancos:

```
13 depara = {}
14 for registro in dados:
15     cnpj = registro.get("CnpjInstituicaoLider")
16     nome = registro.get("NomeInstituicao")
17     if cnpj and nome:
18         cnpj_normalizado = cnpj.zfill(8)
19         depara[cnpj_normalizado] = nome
20
21 print(f"Total de CNPJs únicos no de-para: {len(depara)}")
22
23 with open("data/raw/depara_bcb.json", "w", encoding="utf-8") as f:
24     json.dump(depara, f, ensure_ascii=False, indent=2)
25
26 print("De-para salvo em data/raw/depara_bcb.json")
```

Concluído:

```
(base) camil@MacBook-Air atividade_02 % python scripts/monta_depara_bcb.py
Total de registros recebidos: 5744
{'CodInst': 'C0041856', 'Data': '202412', 'NomeInstituicao': 'ABC-BRASIL', 'DataInicioAtividade': 180001, 'Tcb': 'B1', 'Td': 'C',
'Tc': 3, 'SegmentoTb': None, 'Atividade': None, 'Uf': 'SP', 'Municipio': 'São Paulo', 'Sr': 'S3', 'CodConglomeradoFinanceiro': 'C0041856', 'CodConglomeradoPrudencial': 'C0080312', 'CnpjInstituicaoLider': '28195667', 'Situacao': 'A'}
Total de CNPJs únicos no de-para: 631
De-para salvo em data/raw/depara_bcb.json
(base) camil@MacBook-Air atividade_02 %
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Script para criação da tabela de bancos limpa e corrigida, para ser colocada na camada trusted:

```
eEDB-022 > atividade_02 > scripts > tratamento_bancos.py > ...
1  import pandas as pd
2  import json
3  import unicodedata
4
5  # carregar o de-para de cnpj -> nome oficial
6  with open("data/raw/depara_bcb.json", encoding="utf-8") as f:
7      depara = json.load(f)
8
9  # 2. ler o tsv original
10 df = pd.read_csv("data/raw/Bancos/EnquadramentoInicia_v2.tsv", sep="\t", dtype=str)
11
12 print(f"Total de linhas lidas: {len(df)}")
13 print(f"Valores nulos por coluna:\n{df.isnull().sum()}")
14
15 # 3. normalizar o cnpj (zero à esquerda, 8 dígitos)
16 df["CNPJ"] = df["CNPJ"].str.zfill(8)
17
18 # 4. identificar linhas com nome quebrado
19 df["nome_quebrado"] = df["Nome"].str.contains("\ufffd", na=False)
20 print(f"Linhas com nome quebrado antes da correção: {df['nome_quebrado'].sum()}")
21
22 # 5. corrigir os nomes usando o de-para (quando encontrado)
23 def corrigir_nome(row):
24     if row["nome_quebrado"] and row["CNPJ"] in depara:
25         return depara[row["CNPJ"]]
26     return row["Nome"]
27
28 df["Nome"] = df.apply(corrigir_nome, axis=1)
29
30 # 6. verificar quantos ainda ficaram quebrados (cnpj não encontrado no de-para)
31 df["nome_ainda_quebrado"] = df["Nome"].str.contains("\ufffd", na=False)
32 print(f"Linhas com nome ainda quebrado após correção: {df['nome_ainda_quebrado'].sum()}")
33
34 # 7. tratar nulos remanescentes
35 df["Nome"] = df["Nome"].fillna("NAO_INFORMADO")
36 df["Segmento"] = df["Segmento"].fillna("NAO_INFORMADO")
37
38 # 8. remover colunas auxiliares que criamos só para o diagnóstico
39 df = df.drop(columns=["nome_quebrado", "nome_ainda_quebrado"])
40
41 # 9. salvar como parquet na camada trusted
42 df.to_parquet("data/trusted/bancos_trusted.parquet", index=False)
43
44 print(f"Arquivo salvo em data/trusted/bancos_trusted.parquet com {len(df)} linhas")
```

Não deu certo :(

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Só 14 registros foram corrigidos, muito longo do que é necessário:

```
((base) camil@MacBook-Air atividade_02 % python scripts/tratamento_bancos.py
Total de linhas lidas: 1474
Valores nulos por coluna:
Segmento      0
CNPJ          0
Nome          0
dtype: int64
Linhas com nome quebrado antes da correção: 950
Linhas com nome ainda quebrado após correção: 936
Arquivo salvo em data/trusted/bancos_trusted.parquet com 1474 linhas
```

Tentando com outro endpoint da API (EntidadesSupervisionadas), parece mais promissor:

```
((base) camil@MacBook-Air atividade_02 % python scripts/monta_depara_bcb.py
Total de registros recebidos: 5636
Total de CNPJs únicos no de-para: 5576
De-para salvo em data/raw/depara_bcb.json
(base) camil@MacBook-Air atividade_02 %
```

Parece que foi!!!!:

```
((base) camil@MacBook-Air atividade_02 % python scripts/tratamento_bancos.py
Total de linhas lidas: 1474
Valores nulos por coluna:
Segmento      0
CNPJ          0
Nome          0
dtype: int64
Linhas com nome quebrado antes da correção: 950
Linhas com nome ainda quebrado após correção: 0
Arquivo salvo em data/trusted/bancos_trusted.parquet com 1474 linhas
(base) camil@MacBook-Air atividade_02 %
```


Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Funcionou, acentos agora aparecem normal, sem quebras, além disso, os zero a esquerda foram adicionados (inclusive no banco do brasil!):

```
(base) camil@MacBook-Air atividade_02 % python scripts/ver_parquet.py
(1474, 3)
  Segmento      CNPJ                                     Nome
0         S1  00000000          BANCO DO BRASIL – PRUDENCIAL
1         S1  60746948          BRADESCO – PRUDENCIAL
2         S1  30306294          BTG PACTUAL – PRUDENCIAL
3         S1  00360305    CAIXA ECONOMICA FEDERAL – PRUDENCIAL
4         S1  60872504          ITAU – PRUDENCIAL
5         S1  90400888          SANTANDER – PRUDENCIAL
6         S2  92702067          BANRISUL – PRUDENCIAL
7         S2  07237373    BANCO DO NORDESTE DO BRASIL S.A. – PRUDENCIAL
8         S2  33657248          BNDES – PRUDENCIAL
9         S2  33479023          CITIBANK – PRUDENCIAL
  Segmento      CNPJ                                     Nome
35         S3  80230774    COOPERATIVA CENTRAL DE CREDITO E INVESTIMENTO ...
57         S4  03584906          AGÊNCIA DE FOMENTO DO PARANÁ S.A.
248        S4  82527557    COOPERATIVA DE CRÉDITO, POUPANÇA E INVESTIMENT...
267        S4  79086997    COOPERATIVA DE CRÉDITO, POUPANÇA E INVESTIMENT...
529        S5  07026923    COOPERATIVA DE CRÉDITO DE LIVRE ADMISSÃO SICOO...
689        S5  24799033    COOPERATIVA DE CRÉDITO DE LIVRE ADMISSÃO DO VA...
690        S5  01637949    COOPERATIVA DE CRÉDITO DE LIVRE ADMISSÃO DO VA...
713        S5  25387671    COOPERATIVA DE CREDITO DO ALTO PARANAIBA E REG...
719        S5  81011686    COOPERATIVA DE CRÉDITO DO NORTE CATARINENSE E ...
784        S5  81115149    COOPERATIVA DE CRÉDITO, POUPANÇA E INVESTIMENT...
786        S5  81206039    COOPERATIVA DE CRÉDITO, POUPANÇA E INVESTIMENT...
790        S5  77984870    COOPERATIVA DE CRÉDITO, POUPANÇA E INVESTIMENT...
814        S5  06078926    COOPERATIVA DE CRÉDITO, POUPANÇA E INVESTIMENT...
844        S5  04886317    COOPERATIVA DE CRÉDITO, POUPANÇA E INVESTIMENT...
895        S5  07925729    COOPERATIVA DE CRÉDITO E INVESTIMENTO COM INTE...
962        S5  04273001    COOPERATIVA DE CREDITO RURAL COM INTERACAO SOL...
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Empregados

Pegamos as duas bases de empregados: match_v2.csv, com 34 empresas, e o match_less_v2.csv, com só 5 empresas e match mais baixo (entre 65% e 77%).

Essas 5 foram analisadas uma por uma para ver se fazia sentido, e percebemos que todas elas já estavam presentes dentro do próprio match_v2.csv, ou seja, o match_less_v2.csv não trazia nenhuma empresa nova, só repetia o que já existia na base maior (em 4 dos 5 casos com os mesmos dados; no quinto caso, "Banco Alfa", com um match diferente e pior do que o já existente no match_v2.csv). Essa checagem só foi possível de fazer de forma manual/comparativa por causa da baixa quantidade de dados em ambas as tabelas (34 e 5 linhas).

Por isso, decidimos usar apenas o match_v2.csv como base de empregados, sem necessidade de unir com o match_less_v2.csv. Dentro dele, identificamos dois problemas: um match que não fazia sentido, "Apex Group" tava linkado com "APE POUPEX" (70%), e uma duplicata de instituição, já que "JPMorgan Chase & Co" e "J.P. Morgan" são duas páginas diferentes do Glassdoor mas as duas casaram com o mesmo nome oficial, "JP MORGAN CHASE". Decidimos manter só "JPMorgan Chase & Co", por ter mais reviews (22.000 contra 859), e tirar as outras duas linhas da base, para manter a confiabilidade dos nossos dados (será feito via código para manter reprodutibilidade).

Os arquivos vêm com o separador | (pipe), assim como já tinha sido notado na Atividade 1.

Script tratando os dados e salvando em parquet na nossa camada Trusted:

```
eEDB-022 > atividade_02 > scripts > empregados > tratamento_empregados.py > ...
1  import pandas as pd
2
3  # 1. ler apenas o match_v2, que já contém todas as empresas
4  df = pd.read_csv("data/raw/empregados/glassdoor_consolidado_join_match_v2.csv", sep="|", dtype=str)
5  print(f"match_v2: {df.shape[0]} linhas, {df.shape[1]} colunas")
6
7  # 2. remover os matches identificados como incorretos ou duplicados
8  empresas_excluidas = ["Apex Group", "J.P. Morgan"] # Apex Group: match incorreto | J.P. Morgan: duplicata do JPMorg
9  df = df[~df["employer_name"].isin(empresas_excluidas)]
10 print(f"Total após exclusão: {df.shape[0]} linhas")
11
12 # 3. tratar nulos em employer-founded
13 print(f"Nulos em employer-founded antes: {df['employer-founded'].isnull().sum()}")
14 df["employer-founded"] = df["employer-founded"].fillna("NAO_INFORMADO")
15
16 # 4. salvar como parquet na camada trusted
17 df.to_parquet("data/trusted/empregados_trusted.parquet", index=False)
18 print(f"Arquivo salvo em data/trusted/empregados_trusted.parquet com {len(df)} linhas")
19
```

Sucesso na execução:

```
(base) camil@MacBook-Air atividade_02 % python scripts/empregados/tratamento_empregados.py
match_v2: 34 linhas, 23 colunas
Total após exclusão: 32 linhas
Nulos em employer-founded antes: 6
Arquivo salvo em data/trusted/empregados_trusted.parquet com 32 linhas
(base) camil@MacBook-Air atividade_02 %
```


Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Validação da base:

```
((base) camil@MacBook-Air atividade_02 % python scripts/empregados/ver_parquet.py
(32, 23)

    employer_name                               Nome match_percent
0          BNP Paribas                        BNP PARIBAS           100
1          BTG Pactual                        BTG PACTUAL             100
2          Banco Alfa                          ALFA                   100
3          Banco BMG                           BMG                     100
4          Banco Bradesco                      BRADESCO                100
5          Banco Daycoval                      BANCO DAYCOVAL S.A      100
6          Banco Fibra                        BANCO FIBRA S.A.        100
7          Banco Original                      ORIGINAL                100
8          Banco PAN                           PAN                     100
9          Banco Pine                          PINE                    100
10         Banco Votorantim                    VOTORANTIM              100
11         Banco do Brasil                     BANCO DO BRASIL         100
12         Banco do Nordeste do Brasil         BANCO DO NORDESTE DO BRASIL S.A. 100
13         Bank Of Tokyo-Mitsubishi UFJ        BANCO TOKYO-MITSUBISHI BM S.A.    80
14         Banrisul                           BANRISUL                 100
15         Caixa Econômica Federal             CAIXA ECONOMICA FEDERAL    98
16         Citi                               CITIBANK                 100
17         Credit Suisse                      CREDIT SUISSE            100
18         Deutsche Bank                     DEUTSCHE                 100
19         Goldman Sachs                      GOLDMAN SACHS            100
20         ING                               ING                       100
21         Itau                               ITAU                     100
22         JPMorgan Chase & Co                 JP MORGAN CHASE          88
23         Merrill Lynch                      BOFA MERRILL LYNCH        84
24         Morgan Stanley                     MORGAN STANLEY           100
25         Santander                          SANTANDER                 100
26         Sicredi                            SICREDI                   100
27         Société Générale                    SOCIETE GENERALE         86
28         Votorantim                          VOTORANTIM               100
29         China Construction Bank             CHINA CONSTRUCTION BANK (BRASIL) BANCO MULTIPLO S/    77
30         Mercedes-Benz Mobility              MERCEDES-BENZ            73
31         Sumitomo Mitsui Financial Group     BANCO SUMITOMO MITSUI BRASIL S.A. 65
(base) camil@MacBook-Air atividade_02 %
```

Tudo certo!

Reclamações

Nas bases de Reclamações (Ano/Trimestre), achamos 8 arquivos, um por trimestre entre 2021 e 2022. O arquivo 2022_tri_02_nao_ha_dados.csv veio vazio e não vai entrar no tratamento (o próprio nome já avisa isso). Os outros 7 têm a mesma estrutura, com 15 colunas, e cada um já vem com o Ano e Trimestre identificados dentro dos dados, então dá pra empilhar todos numa base só sem problema.

Alguns problemas que encontramos nesses arquivos:

O cabeçalho de algumas colunas veio com caractere quebrado, como em "Quantidade total de clientes \x96 CCS e SCR", onde o \x96 devia ser um travessão. Essas colunas vão ser renomeadas.

Tem uma coluna extra, chamada Unnamed: 14, que vem vazia em todos os arquivos, provavelmente por causa de um ; a mais no final de cada linha. Essa coluna vai ser removida.

O campo CNPJ IF vem vazio quando o registro é do tipo Conglomerado, porque conglomerado é um grupo de instituições e não tem CNPJ próprio, só as instituições individuais (tipo Banco/financeira) têm. Isso não é exatamente um erro, é assim mesmo que os dados funcionam, mas vamos preencher com "NAO_SE_APLICA" pra deixar claro que é esperado.

O campo Índice também vem vazio (só espaço em branco) em alguns casos, quando não deu pra calcular. Vamos tratar como nulo/zero.

Os números vêm no formato brasileiro, com vírgula no lugar do ponto (tipo "54,79"), e precisam ser convertidos pra formato numérico normal antes de fazer qualquer cálculo com eles.

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Script tratando os dados e salvando em parquet na nossa camada Trusted:

```
eEDB-022 > atividade_02 > scripts > reclamacoes > tratamento_reclamacoes.py > ...
1  import pandas as pd
2  import glob
3
4  # 1. listar todos os csvs de reclamacoes, exceto o vazio
5  arquivos = sorted(glob.glob("data/raw/Reclamacoes/*.csv"))
6  arquivos = [a for a in arquivos if "nao_ha_dados" not in a]
7  print(f"Arquivos a processar: {len(arquivos)}")
8  for a in arquivos:
9      print("-", a)
10
11 # 2. ler e empilhar todos os trimestres
12 dfs = []
13 for arq in arquivos:
14     df_temp = pd.read_csv(arq, sep=";", encoding="latin1", dtype=str)
15     dfs.append(df_temp)
16
17 df = pd.concat(dfs, ignore_index=True)
18 print(f"Total após empilhar todos os trimestres: {df.shape[0]} linhas")
19
20 # 3. remover a coluna fantasma (Unnamed: 14)
21 df = df.loc[:, ~df.columns.str.startswith("Unnamed")]
22
23 # 4. corrigir nomes de colunas com caractere quebrado (\x96 = travessão)
24 df.columns = [c.replace("\x96", "-") for c in df.columns]
25 print("Colunas finais:", list(df.columns))
26
27 # 5. tratar CNPJ vazio (esperado para linhas tipo Conglomerado)
28 df["CNPJ IF"] = df["CNPJ IF"].str.strip()
29 df["CNPJ IF"] = df["CNPJ IF"].replace("", "NAO_SE_APLICA")
30
31 # 6. tratar Índice: remover separador de milhar, vírgula -> ponto, vazio -> nulo, converter para número
32 df["Índice"] = df["Índice"].str.strip()
33 df["Índice"] = df["Índice"].replace("", None)
34 df["Índice"] = df["Índice"].str.replace(".", "", regex=False)
35 df["Índice"] = df["Índice"].str.replace(",", ".", regex=False)
36 df["Índice"] = df["Índice"].astype(float)
37 print(f"Nulos em Índice após tratamento: {df['Índice'].isnull().sum()}")
38
39 # 7. salvar como parquet na camada trusted
40 df.to_parquet("data/trusted/reclamacoes_trusted.parquet", index=False)
41 print(f"Arquivo salvo em data/trusted/reclamacoes_trusted.parquet com {len(df)} linhas")
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Execução

```
(base) camil@MacBook-Air atividade_02 % python scripts/reclamacoes/tratamento_reclamacoes.py
Arquivos a processar: 7
- data/raw/Reclamacoes/2021_tri_01.csv
- data/raw/Reclamacoes/2021_tri_02.csv
- data/raw/Reclamacoes/2021_tri_03.csv
- data/raw/Reclamacoes/2021_tri_04.csv
- data/raw/Reclamacoes/2022_tri_01.csv
- data/raw/Reclamacoes/2022_tri_03.csv
- data/raw/Reclamacoes/2022_tri_04.csv
Total após empilhar todos os trimestres: 918 linhas
Colunas finais: ['Ano', 'Trimestre', 'Categoria', 'Tipo', 'CNPJ IF', 'Instituição financeira', 'Índice', 'Quantidade de reclamações reguladas procedentes', 'Quantidade de reclamações reguladas - outras', 'Quantidade de reclamações não reguladas', 'Quantidade total de reclamações', 'Quantidade total de clientes - CCS e SCR', 'Quantidade de clientes - CCS', 'Quantidade de clientes - SCR']
Nulos em Índice após tratamento: 651
Arquivo salvo em data/trusted/reclamacoes_trusted.parquet com 918 linhas
(base) camil@MacBook-Air atividade_02 %
```

OBS:

Os 7 arquivos trimestrais (2021 e 2022) foram empilhados em uma base única de 918 linhas. O campo Índice ficou nulo em 651 linhas (71%). Investigando o motivo, descobrimos que o Banco Central só calcula esse índice pras instituições maiores (categorias Top10 e Top15), então pras demais instituições esse dado realmente não existe na fonte, não é falha de leitura. Como Índice é um campo numérico, não fazia sentido colocar um texto tipo "não se aplica" nele (perderia a possibilidade de fazer conta com a coluna depois). Por isso, decidimos criar uma coluna nova, `indice_disponivel`, marcando True ou False, pra deixar explícito quando o índice não existe na fonte, mantendo o campo Índice como número mesmo com os nulos.

Antes:

```
# 6. tratar Índice: remover separador de milhar, vírgula -> ponto, vazio -> nulo, converter para número
df["Índice"] = df["Índice"].str.strip()
df["Índice"] = df["Índice"].replace("", None)
df["Índice"] = df["Índice"].str.replace(".", "", regex=False)
df["Índice"] = df["Índice"].str.replace(",", ".", regex=False)
df["Índice"] = df["Índice"].astype(float)
print(f"Nulos em Índice após tratamento: {df['Índice'].isnull().sum()}")
```

Depois:

```
# 6. tratar indice: sinalizar disponibilidade, remover separador de milhar, vírgula -> ponto, converter para número
df["Índice"] = df["Índice"].str.strip()
df["indice_disponivel"] = df["Índice"] != ""
df["Índice"] = df["Índice"].replace("", None)
df["Índice"] = df["Índice"].str.replace(".", "", regex=False)
df["Índice"] = df["Índice"].str.replace(",", ".", regex=False)
df["Índice"] = df["Índice"].astype(float)
print(f"Linhas com indice_disponivel = False: {(~df['indice_disponivel']).sum()}")
```

No tratamento do campo Índice, adicionamos uma linha logo após o `strip()` (remoção de espaços), antes de qualquer conversão de valor: a coluna `indice_disponivel`, que marca True quando o índice veio preenchido na fonte e False quando veio vazio. Essa checagem precisa vir antes do `replace("", None)` que transforma os vazios em nulo, senão perderíamos a informação de quem era vazio originalmente. O restante do tratamento (remover separador de milhar, trocar vírgula por ponto, converter pra número) continua igual. No final, trocamos também o print de contagem: em vez de contar os nulos direto na coluna Índice, passamos a contar pela nova coluna `indice_disponivel`, que fica registrada no arquivo final como explicação do motivo de cada ausência.

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Execução:

```
(base) camil@MacBook-Air atividade_02 % python scripts/reclamacoes/tratamento_reclamacoes.py
Arquivos a processar: 7
- data/raw/Reclamacoes/2021_tri_01.csv
- data/raw/Reclamacoes/2021_tri_02.csv
- data/raw/Reclamacoes/2021_tri_03.csv
- data/raw/Reclamacoes/2021_tri_04.csv
- data/raw/Reclamacoes/2022_tri_01.csv
- data/raw/Reclamacoes/2022_tri_03.csv
- data/raw/Reclamacoes/2022_tri_04.csv
Total após empilhar todos os trimestres: 918 linhas
Colunas finais: ['Ano', 'Trimestre', 'Categoria', 'Tipo', 'CNPJ IF', 'Instituição financeira', 'Índice', 'Quantidade de reclamações reguladas procedentes', 'Quantidade de reclamações reguladas - outras', 'Quantidade de reclamações não reguladas', 'Quantidade total de reclamações', 'Quantidade total de clientes - CCS e SCR', 'Quantidade de clientes - CCS', 'Quantidade de clientes - SCR']
Linhas com índice_disponível = False: 651
Arquivo salvo em data/trusted/reclamacoes_trusted.parquet com 918 linhas
(base) camil@MacBook-Air atividade_02 %
```

Deu certo!

```
(base) camil@MacBook-Air atividade_02 % python scripts/reclamacoes/ver_parquet.py
(918, 15)
  Ano Trimestre  Instituição financeira  CNPJ IF  Tipo  Índice
0  2021      1º  ABC-BRASIL (conglomerado)  NAO_SE_APLICA  Conglomerado  NaN
1  2021      1º  AGIBANK (conglomerado)  NAO_SE_APLICA  Conglomerado  54.79
2  2021      1º  AGORACRED S/A SOCIEDADE DE CRÉDITO, FINANCIAMENTO E INVESTIMENTO  36321990  Banco/financeira  NaN
3  2021      1º  AL5 S.A. CRÉDITO, FINANCIAMENTO E INVESTIMENTO  27214112  Banco/financeira  NaN
4  2021      1º  ALFA (conglomerado)  NAO_SE_APLICA  Conglomerado  NaN
5  2021      1º  ANDBANK (conglomerado)  NAO_SE_APLICA  Conglomerado  NaN
6  2021      1º  BANCO ABN AMRO S.A.  03532415  Banco/financeira  NaN
7  2021      1º  BANCO ARBI S.A.  54403563  Banco/financeira  NaN
8  2021      1º  BANCO CNH INDUSTRIAL CAPITAL S.A.  02992446  Banco/financeira  NaN
9  2021      1º  BANCO CSF S.A.  08357240  Banco/financeira  22.53
10 2021      1º  BANCO DA AMAZONIA S.A.  04902979  Banco/financeira  NaN
11 2021      1º  BANCO DA CHINA BRASIL S.A.  10690848  Banco/financeira  NaN
12 2021      1º  BANCO DE LAGE LANDEN BRASIL S.A.  05040481  Banco/financeira  NaN
13 2021      1º  BANCO DIGIMAI S.A.  92874270  Banco/financeira  484.31
14 2021      1º  BANCO DIGIO S.A.  27098060  Banco/financeira  25.71

Distribuição por Ano/Trimestre:
Ano Trimestre
2021 1º      105
     2º      111
     3º      113
     4º      135
2022 1º      137
     3º      163
     4º      154
dtype: int64
```

Tabela final

Granularidades:

Bancos: granularidade de uma linha por instituição financeira, sem repetição.

Empregados: também uma linha por empresa, sem repetição.

Reclamações: granularidade de uma linha por instituição, por trimestre. A mesma instituição aparece várias vezes na tabela, uma vez pra cada trimestre em que teve reclamações registradas.

A ideia é conseguir cruzar a nota geral da empresa no Glassdoor (satisfação dos colaboradores) com o volume/índice de reclamações de clientes registradas no Banco Central, pra ver se existe alguma relação entre elas: bancos com colaboradores mais satisfeitos têm menos reclamações de clientes ou não?

Pra montar essa tabela final, vamos seguir 3 passos:

1. Pegar a base de Bancos como ponto de partida, já que é ela que vai virar a dimensão (uma linha por instituição).
2. Juntar com Empregados usando o campo Nome, trazendo as notas do Glassdoor (nota geral, remuneração, liderança, etc.) pra dentro da linha de cada banco.
3. Juntar com Reclamações usando o CNPJ, mas antes disso, resumir os 7 trimestres de cada banco em um número só: por exemplo, somar o total de reclamações do período inteiro e calcular a média do Índice. Assim cada banco fica com uma linha só, mesmo tendo várias reclamações trimestrais por trás.

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Script:

```
eEDB-022 > atividade_02 > scripts > delivery > tratamento_delivery.py > ...
1  import pandas as pd
2
3  # 1. ler as tres tabelas trusted
4  bancos = pd.read_parquet("data/trusted/bancos_trusted.parquet")
5  empregados = pd.read_parquet("data/trusted/empregados_trusted.parquet")
6  reclamacoes = pd.read_parquet("data/trusted/reclamacoes_trusted.parquet")
7
8  print(f"Bancos: {bancos.shape[0]} linhas")
9  print(f"Empregados: {empregados.shape[0]} linhas")
10 print(f"Reclamacoes: {reclamacoes.shape[0]} linhas")
11
12 # 2. criar coluna auxiliar de nome normalizado, so para o join (nao mexe no Nome original)
13 bancos["nome_join"] = bancos["Nome"].str.replace(" - PRUDENCIAL", "", regex=False).str.strip()
14 empregados["nome_join"] = empregados["Nome"].str.strip()
15
16 # 3. juntar bancos com empregados, usando o campo nome_join (match exato)
17 colunas_glassdoor = [
18     "nome_join", "employer_name", "Geral", "Cultura e valores",
19     "Diversidade e inclusão", "Qualidade de vida", "Alta liderança",
20     "Remuneração e benefícios", "Oportunidades de carreira",
21     "reviews_count", "match_percent"
22 ]
23 df = bancos.merge(empregados[colunas_glassdoor], on="nome_join", how="left")
24 print(f"Apos juntar com Empregados: {df.shape[0]} linhas")
25 print(f"Bancos com dado do Glassdoor: {df['Geral'].notnull().sum()}")
26
27 # 4. agregar reclamacoes por cnpj (resumindo os 7 trimestres em uma linha por banco)
28 agregado_reclamacoes = reclamacoes.groupby("CNPJ IF").agg(
29     total_reclamacoes=("Quantidade total de reclamações", lambda x: x.astype(float).sum()),
30     indice_medio=("Índice", "mean"),
31     trimestres_com_reclamacao=("Trimestre", "count")
32 ).reset_index()
33 agregado_reclamacoes = agregado_reclamacoes.rename(columns={"CNPJ IF": "CNPJ"})
34 print(f"Instituicoes com reclamacoes agregadas: {agregado_reclamacoes.shape[0]}")
35
36 # 5. juntar o resultado com o agregado de reclamacoes, usando cnpj
37 df = df.merge(agregado_reclamacoes, on="CNPJ", how="left")
38 print(f"Apos juntar com Reclamacoes: {df.shape[0]} linhas")
39 print(f"Bancos com dado de reclamacoes: {df['total_reclamacoes'].notnull().sum()}")
40
41 # 6. remover coluna auxiliar que criamos so para o join
42 df = df.drop(columns=["nome_join"])
43
44 # 7. salvar como parquet na camada delivery
45 df.to_parquet("data/trusted/delivery_bancos.parquet", index=False)
46 print(f"Arquivo salvo com {len(df)} linhas e {len(df.columns)} colunas")
```

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Execução:

```
(base) camil@MacBook-Air atividade_02 % python scripts/delivery/tratamento_delivery.py
Bancos: 1474 linhas
Empregados: 31 linhas
Reclamacoes: 918 linhas
Apos juntar com Empregados: 1474 linhas
Bancos com dado do Glassdoor: 31
Instituicoes individuais com reclamacoes agregadas: 112
Conglomerados com reclamacoes agregadas: 95
Apos juntar com Reclamacoes (individual + conglomerado): 1474 linhas
Bancos com dado de reclamacoes: 119
Bancos com Glassdoor E Reclamacoes: 23
Arquivo salvo com 1474 linhas e 16 colunas
```

```
(base) camil@MacBook-Air atividade_02 % python scripts/delivery/ver_parquet.py
(1474, 16)
['Segmento', 'CNPJ', 'Nome', 'employer_name', 'Geral', 'Cultura e valores', 'Diversidade e inclusão', 'Qualidade de vida', 'Alta liderança', 'Remuneração e benefícios', 'Oportunidades de carreira', 'reviews_count', 'match_percent', 'total_reclamacoes', 'indice_medio', 'trimestres_com_reclamacao']

Bancos com Glassdoor E Reclamacoes: 23
```

	Nome Geral	total_reclamacoes	indice_medio
1	BRDESCO - PRUDENCIAL 4.2	60835.0	25.925714
2	BTG PACTUAL - PRUDENCIAL 4.0	133.0	NaN
4	ITAU - PRUDENCIAL 4.5	43280.0	15.291429
5	SANTANDER - PRUDENCIAL 3.9	45423.0	27.284286
6	BANRISUL - PRUDENCIAL 4.2	3689.0	21.584286
7	BANCO DO NORDESTE DO BRASIL S.A. - PRUDENCIAL 4.5	490.0	NaN
9	CITIBANK - PRUDENCIAL 3.9	97.0	NaN
10	CREDIT SUISSE - PRUDENCIAL 3.8	4.0	NaN
12	VOTORANTIM - PRUDENCIAL 4.4	4529.0	10.640000
14	ALFA - PRUDENCIAL 3.5	480.0	NaN
24	BANCO FIBRA S.A. - PRUDENCIAL 4.3	17.0	NaN
26	BANCO SUMITOMO MITSUI BRASIL S.A. - PRUDENCIAL 3.4	2.0	NaN
29	BMG - PRUDENCIAL 4.1	12161.0	84.215714
30	BNP PARIBAS - PRUDENCIAL 3.8	4624.0	112.770000
31	BOFA MERRILL LYNCH - PRUDENCIAL 3.8	7.0	NaN
37	DEUTSCHE - PRUDENCIAL 3.9	1.0	NaN
42	JP MORGAN CHASE - PRUDENCIAL 4.0	9.0	NaN
44	MERCEDES-BENZ - PRUDENCIAL 4.1	35.0	NaN
46	ORIGINAL - PRUDENCIAL 3.8	10168.0	37.840000
47	PAN - PRUDENCIAL 4.4	3953.0	245.280000
48	PINE - PRUDENCIAL 3.8	29.0	NaN
49	SICREDI - PRUDENCIAL 4.4	18.0	NaN
50	SOCIETE GENERALE - PRUDENCIAL 3.7	6.0	NaN

```
(base) camil@MacBook-Air atividade_02 %
```

Conseguimos juntar as três fontes numa tabela só, com 23 bancos tendo tanto a nota do Glassdoor quanto o total de reclamações. Olhando os números soltos, não dá pra ver um padrão óbvio: o Itaú tem nota alta (4.5) mas também tem reclamação alta (43.280), enquanto o BTG Pactual tem nota parecida (4.0) mas reclamação bem menor (133). Já o PAN tem nota boa (4.4) mas o maior índice médio de todos (245.28).

Isso sugere que o total de reclamações parece acompanhar mais o tamanho do banco (quantos clientes ele tem) do que a nota de satisfação dos funcionários, já que bancos grandes como Itaú, Bradesco e Santander aparecem com os maiores volumes de reclamação, independente da nota.

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados e EDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Subindo para o PostgreSQL

Script de subida:

```
eEDB-022 > atividade_02 > scripts > delivery > subir_postgres.py > ...
1  import pandas as pd
2  import sys
3  sys.path.append("scripts")
4  from conexao import get_engine
5
6  df = pd.read_parquet("data/trusted/delivery_bancos.parquet")
7
8  engine = get_engine()
9  df.to_sql("delivery_bancos", engine, if_exists="replace", index=False)
10
11  print(f"Tabela 'delivery_bancos' criada no Postgres com {len(df)} linhas")
12
13
```

```
((base) camil@MacBook-Air atividade_02 % python scripts/delivery/subir_postgres.py
Tabela 'delivery_bancos' criada no Postgres com 1474 linhas
(base) camil@MacBook-Air atividade_02 %
```

```
((base) camil@MacBook-Air atividade_02 % psql -h localhost -p 5433 -U postgres -d eedb022_a2
Password for user postgres:
psql (18.0, server 16.14 (Debian 16.14-1.pgdg13+1))
Type "help" for help.

eedb022_a2=# SELECT COUNT(*) FROM delivery_bancos;
 count
-----
  1474
(1 row)

eedb022_a2=# SELECT "Nome", "Geral", total_reclamacoes FROM delivery_bancos WHERE "Geral" IS NOT NULL LIMIT 10;
      Nome      | Geral | total_reclamacoes
-----|-----|-----
BANCO DO BRASIL - PRUDENCIAL | 4.3   |
BRADESCO - PRUDENCIAL | 4.2   | 60835
BTG PACTUAL - PRUDENCIAL | 4.0   | 133
CAIXA ECONOMICA FEDERAL - PRUDENCIAL | 4.4   |
ITAU - PRUDENCIAL | 4.5   | 43280
SANTANDER - PRUDENCIAL | 3.9   | 45423
BANRISUL - PRUDENCIAL | 4.2   | 3689
BANCO DO NORDESTE DO BRASIL S.A. - PRUDENCIAL | 4.5   | 490
CITIBANK - PRUDENCIAL | 3.9   | 97
CREDIT SUISSE - PRUDENCIAL | 3.8   | 4
(10 rows)

eedb022_a2=#
```

Tabela no banco!

OBS:

A tabela Delivery final ficou com 1.474 linhas, que é a base completa de Bancos (a dimensão inteira, com todas as instituições). Dessas 1.474, só 23 bancos têm ao mesmo tempo nota do Glassdoor e dado de reclamações preenchidos.

Isso é esperado, não é falha do cruzamento. A base de Empregados (Glassdoor) tem só 31 empresas no total, e a base de Reclamações tem 113 instituições reportadas nos trimestres analisados. Ou seja, mesmo que o cruzamento fosse perfeito, o máximo possível de bancos com as duas métricas juntas já seria limitado pelo tamanho da menor base (31, do Glassdoor).

Curso de Especialização em Big Data – Escola Politécnica da USP

Ingestão de Dados eEDB-022

Atividade 1

Grupo: Camila Faleiros, Fernando Luiz, Guilherme Sergio e Lucca Tomazeli

Conseguir 23 de um máximo teórico de 31 é uma taxa de acerto de quase 75%, dentro do que era possível com os dados disponíveis.

Ou seja, o "gargalo" não é problema no processo de tratamento, é que as bases de origem em si são pequenas e não cobrem as mesmas instituições. As outras linhas continuam na tabela final, só que com essas colunas extras vazias, porque decidimos manter todos os bancos na base, em vez de descartar quem não tinha as informações extras.